Clasificación en PyTorch

Guía académica 02 · De tensores y *autograd* a `torch.nn` y PyTorch Lightning

Juan Sebastián Laverde Chunza

15 de octubre de 2026

Esta guía construye un clasificador binario en PyTorch de abajo hacia arriba. Empieza por los tensores, los conjuntos de datos y la entropía cruzada binaria, que se deriva y se programa a mano. Luego usa `autograd` para el descenso de gradiente y organiza la red con `torch.nn`. Incluye capas propias, un bucle de entrenamiento al estilo de Keras con *early stopping* y puntos de control, regularización y normalización. Termina con dos versiones de alto nivel: PyTorch Lightning y Keras 3 sobre el *backend* de PyTorch. Todo el código se ejecutó al generar el documento.

Tabla de contenidos

1 Propósito y mapa de lectura **2**

2 Intuición: una probabilidad que se corrige **3**

3 Fundamentos matemáticos **3**
3.1 De la puntuación a la probabilidad . . . 3
3.2 Entropía cruzada binaria . . . 3
3.3 El gradiente que hace todo simple . . . 3
3.4 Ejemplo numérico resuelto . . . 4

4 Datos: espirales, `Dataset` y `DataLoader` **5**
4.1 Generación del problema . . . 5
4.2 `Dataset`: qué es un ejemplo . . . 6
4.3 `DataLoader`: cómo se sirven los lotes . . . 7

5 Entrenamiento manual: regresión logística con *autograd* **7**

6 `torch.nn`: modelos como módulos **8**
6.1 `nn.Module` . . . 8
6.2 Capas propias: densa y *Dropout* desde cero . . . 8
6.3 `train()` y `eval()` . . . 9

7 Un bucle de entrenamiento reutilizable, al estilo Keras **9**
7.1 La frontera de decisión . . . 11

8 Evaluación final y guardado **12**
8.1 Prueba, matriz de confusión y métricas . . . 12
8.2 Guardar y restaurar . . . 13

9 Regularización y normalización en PyTorch **13**
9.1 L2, L1 y *Dropout* 13
9.2 *Batch*, *Layer* y *Group Normalization* 14

10 PyTorch Lightning: el mismo modelo con menos código de infraestructura **15**

11 Keras 3 con *backend* PyTorch **16**

12 Comparación: PyTorch, Lightning y Keras **17**

13 Dimensiones de tensores y flujo de datos **18**

14 Hiperparámetros del modelo **19**

15 Errores frecuentes **20**

16 Ejercicios **20**
16.1 Respuestas orientativas 21

17 Glosario **21**

18 Conclusiones académicas **21**

19 Referencias **22**

Procedencia y autoría

Autor de esta guía. Juan Sebastián Laverde Chunza, con apoyo de herramientas de inteligencia artificial generativa. Es material académico complementario y **no** es material oficial del docente ni de la Universidad El Bosque.
Referencia académica. Los temas siguen el notebook `02_Clasificacion.ipynb` de la Clase 02 del curso Deep Learning I, del profesor **Héctor Javier Hortúa, PhD** (GitHub · Colab).
Edición pública. El código, los datos, los ejemplos y las explicaciones de esta edición son propios e independientes: no reproducen el código ni el texto del notebook del docente. Citar el notebook como referencia no equivale a una licencia ni a una autorización sobre el material original.

1. Propósito y mapa de lectura

Clasificar es asignar una etiqueta a cada ejemplo: correo legítimo o no deseado, transacción normal o sospechosa, imagen de un 3 o de un 8. En la **clasificación binaria** hay dos clases y el modelo estima la probabilidad de la clase positiva. Esta guía usa un problema geométrico difícil: separar dos espirales entrelazadas. Ninguna recta puede hacerlo, así que el ejemplo obliga a usar una red no lineal.

Bloque	Contenido
§3 Fundamentos	Sigmoide, entropía cruzada binaria, su gradiente y un ejemplo resuelto
§4 Datos	Tensores, `Dataset` y `DataLoader`
§5 Entrenamiento manual	Regresión logística con *autograd* y descenso de gradiente explícito

Bloque	Contenido
§6 `torch.nn`	`nn.Module`, capas propias, MLP y modos `train()/eval()`
§7–§8 Bucle estilo Keras	`fit/evaluate/predict`, historial, *early stopping*, frontera de decisión, métricas, puntos de control y guardado
§9 Generalización	L1, L2, *Dropout*, *Batch/Layer/Group Normalization*
§10–§12 Alto nivel	PyTorch Lightning, Keras 3 con *backend* PyTorch y comparación

2. Intuición: una probabilidad que se corrige

Un clasificador binario funciona como un evaluador que, ante cada caso, da una **probabilidad** en lugar de un «sí» o un «no» rotundo. Si dice 0.9 y acierta, el error es pequeño. Si dice 0.9 y se equivoca, el error es grande, porque estaba muy seguro de algo falso. La entropía cruzada mide exactamente eso: castiga más la **confianza equivocada** que la duda. Entrenar consiste en ajustar los parámetros para que las probabilidades altas recaigan sobre las respuestas correctas.

3. Fundamentos matemáticos

3.1. De la puntuación a la probabilidad

Un modelo produce una puntuación real z, el ***logit***. La **sigmoide** la convierte en probabilidad:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\,(1 - \sigma(z)).$$

La regla de decisión habitual es predecir la clase 1 si $p \geq 0.5$, es decir, si $z \geq 0$.

3.2. Entropía cruzada binaria

Para una etiqueta $y \in \{0, 1\}$:

$$\ell(z, y) = -\big[\, y \log p + (1 - y) \log(1 - p) \,\big], \qquad p = \sigma(z).$$

La expresión sale de la **máxima verosimilitud**. Si y sigue una distribución de Bernoulli con parámetro p, la probabilidad de observar y es $p^y (1 - p)^{1-y}$. Maximizar su logaritmo equivale a minimizar ℓ.

3.3. El gradiente que hace todo simple

Derivando con la regla de la cadena:

$$\frac{\partial \ell}{\partial z} = -\left[\frac{y}{p} - \frac{1 - y}{1 - p}\right] p(1 - p) = p - y.$$

El gradiente respecto al logit es la diferencia entre la probabilidad estimada y la etiqueta. Para una regresión logística $z = \mathbf{w}^\top \mathbf{x} + b$, se cumple $\partial \ell / \partial \mathbf{w} = (p - y)\,\mathbf{x}$ y $\partial \ell / \partial b = p - y$.

Estabilidad numérica

Calcular $\log \sigma(z)$ directamente produce $\log 0 = -\infty$ cuando z es muy negativo. Las bibliotecas usan la forma equivalente $\ell = \max(z, 0) - z\,y + \log\!\left(1 + e^{-|z|}\right)$. Por eso en PyTorch se prefiere `BCEWithLogitsLoss`, que recibe logits, a aplicar la sigmoide y después `BCELoss`.

3.4. Ejemplo numérico resuelto

Sea $\mathbf{x} = (1,\ 2)$, $\mathbf{w} = (0.5,\ -0.25)$, $b = 0.1$ y etiqueta $y = 1$.

1. Logit: $z = 0.5 \cdot 1 - 0.25 \cdot 2 + 0.1 = 0.1$.
2. Probabilidad: $p = \sigma(0.1) = 0.52498$.
3. Pérdida: $\ell = -\log(0.52498) = 0.64440$.
4. Gradientes: $p - y = -0.47502$, de modo que $\nabla_{\mathbf{w}}\ell = (-0.47502,\ -0.95004)$ y $\partial_b \ell = -0.47502$.

El signo negativo indica que aumentar $\mathbf{w}$ en la dirección de $\mathbf{x}$ reduce la pérdida: el modelo debe subir la probabilidad de la clase verdadera.

```
import os
os.environ["KERAS_BACKEND"] = "torch"   # debe fijarse antes de importar keras

import logging, warnings, tempfile, pathlib, copy
warnings.filterwarnings("ignore")
logging.getLogger("lightning.pytorch").setLevel(logging.ERROR)

import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import Dataset, DataLoader, random_split

torch.manual_seed(2026)
np.random.seed(2026)
print("PyTorch", torch.__version__)
```

```
PyTorch 2.14.1+cpu
```

```
x = torch.tensor([1.0, 2.0])
w = torch.tensor([0.5, -0.25], requires_grad=True)
b = torch.tensor(0.1, requires_grad=True)
y = torch.tensor(1.0)

z = w @ x + b
loss = nn.functional.binary_cross_entropy_with_logits(z, y)
loss.backward()
print(f"z={z.item():.4f}  p={torch.sigmoid(z).item():.5f}  "
      f"perdida={loss.item():.5f}")
print("grad w:", w.grad.numpy().round(5), "| grad b:", round(b.grad.item(), 5))
```

```
z=0.1000  p=0.52498  perdida=0.64440
grad w: [-0.47502 -0.95004] | grad b: -0.47502
```

`autograd` reproduce los valores del ejemplo. Al marcar un tensor con `requires_grad=True`, PyTorch registra las operaciones en las que participa. `backward()` recorre ese registro en sentido inverso y acumula las derivadas en el atributo `.grad`.

4. Datos: espirales, `Dataset` y `DataLoader`

4.1. Generación del problema

Cada clase es un brazo de espiral en el plano. El segundo brazo es el primero girado 180 grados, y a ambos se les añade ruido gaussiano.

```
def twin_spirals(points_per_arm=500, turns=1.75, noise=0.18, seed=0):
    gen = np.random.default_rng(seed)
    t = np.sqrt(gen.uniform(0.02, 1.0, points_per_arm)) * turns * 2 * np.pi
    arm = np.stack([t * np.cos(t), t * np.sin(t)], axis=1) / (turns * 2 * np.pi)
    coords = np.concatenate([arm, -arm]) * 2.0
    coords += gen.normal(0.0, noise, coords.shape)
    labels = np.concatenate([np.zeros(points_per_arm), np.ones(points_per_arm)])
    return coords.astype(np.float32), labels.astype(np.float32)

coords, labels = twin_spirals()
print("caracteristicas:", coords.shape, "| etiquetas:", labels.shape,
      "| proporcion clase 1:", labels.mean())
```

```
caracteristicas: (1000, 2) | etiquetas: (1000,) | proporcion clase 1: 0.5
```

```
fig, ax = plt.subplots(figsize=(4.2, 4.0))
for cls, marker in [(0, "o"), (1, "^")]:
    pick = labels == cls
    ax.scatter(coords[pick, 0], coords[pick, 1], s=7, marker=marker,
               alpha=0.6, label=f"clase {cls}")
ax.set_aspect("equal"); ax.legend(loc="upper right")
plt.tight_layout(); plt.show()
```

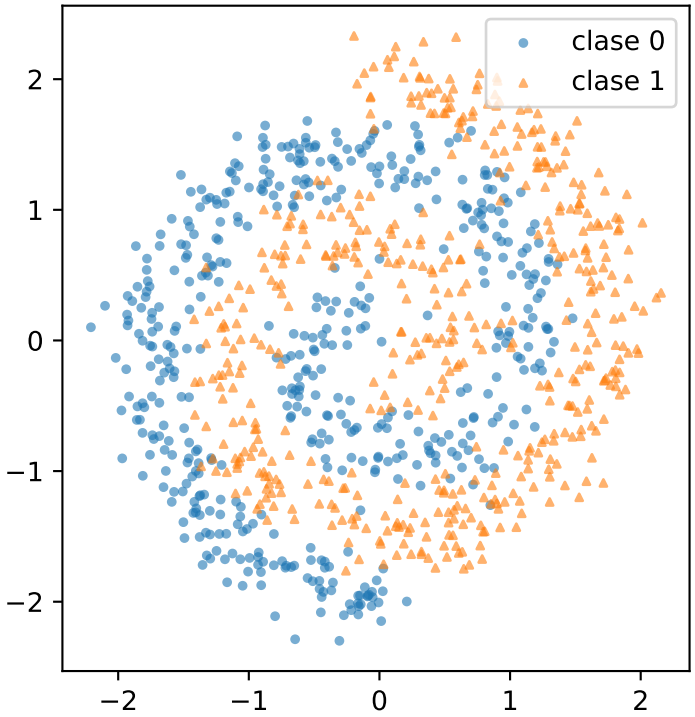


Figura 1: Dos espirales entrelazadas: ninguna recta separa las clases.

4.2. `Dataset`: qué es un ejemplo

Una subclase de `Dataset` responde a dos preguntas: cuántos ejemplos hay (`__len__`) y cuál es el ejemplo número i (`__getitem__`). Aquí también puede ir la lectura desde disco o el aumento de datos.

```
class PointCloud(Dataset):
    def __init__(self, features, targets):
        self.features = torch.as_tensor(features)
        self.targets = torch.as_tensor(targets).unsqueeze(1)

    def __len__(self):
        return len(self.targets)

    def __getitem__(self, index):
        return self.features[index], self.targets[index]

full_set = PointCloud(coords, labels)
splitter = torch.Generator().manual_seed(7)
learn_set, tune_set, hold_set = random_split(full_set, [0.7, 0.15, 0.15],
                                             generator=splitter)
print("entrenamiento:", len(learn_set), "| validacion:", len(tune_set),
      "| prueba:", len(hold_set))
```

```
entrenamiento: 700 | validacion: 150 | prueba: 150
```

4.3. `DataLoader`: cómo se sirven los lotes

`DataLoader` agrupa ejemplos en lotes, los mezcla en cada época si `shuffle=True` y puede usar varios procesos. Solo se mezcla el conjunto de entrenamiento: en validación y prueba el orden no afecta al resultado.

```
learn_loader = DataLoader(learn_set, batch_size=32, shuffle=True,
                          generator=torch.Generator().manual_seed(3))
tune_loader = DataLoader(tune_set, batch_size=256)
hold_loader = DataLoader(hold_set, batch_size=256)

xb, yb = next(iter(learn_loader))
print("lote X:", tuple(xb.shape), xb.dtype, "| lote y:", tuple(yb.shape))
```

```
lote X: (32, 2) torch.float32 | lote y: (32, 1)
```

5. Entrenamiento manual: regresión logística con *autograd*

El punto de partida es una frontera **lineal**. Los parámetros son tensores con `requires_grad=True`. En cada paso: predicción, pérdida, `backward()`, actualización dentro de `torch.no_grad()` (para que la actualización no se registre en el grafo) y puesta a cero de los gradientes, porque PyTorch los **acumula** de un paso al siguiente.

```
weights = torch.zeros(2, 1, requires_grad=True)
bias = torch.zeros(1, requires_grad=True)
step_size = 0.5

def accuracy_from_logits(logits, target):
    return ((logits >= 0).float() == target).float().mean().item()

for epoch in range(40):
    for xb, yb in learn_loader:
        logits = xb @ weights + bias
        batch_loss = nn.functional.binary_cross_entropy_with_logits(logits, yb)
        batch_loss.backward()
        with torch.no_grad():
            weights -= step_size * weights.grad
            bias -= step_size * bias.grad
        weights.grad.zero_()
        bias.grad.zero_()

xt, yt = tune_set[:]
with torch.no_grad():
    lin_acc = accuracy_from_logits(xt @ weights + bias, yt)
print(f"exactitud de validacion de la frontera lineal: {lin_acc:.3f}")
```

```
exactitud de validacion de la frontera lineal: 0.647
```

La recta acierta en algo menos de dos de cada tres puntos, y lanzar una moneda daría 0.5. Era esperable: la regresión logística traza una recta, y las espirales no son separables linealmente. La recta solo aprovecha que cada brazo ocupa más una mitad del plano que la otra. Para seguir el giro hace falta una red con capas ocultas.

6. `torch.nn`: modelos como módulos

6.1. `nn.Module`

Toda red en PyTorch hereda de `nn.Module`. Las capas declaradas como atributos en `__init__` se registran automáticamente: sus parámetros aparecen en `.parameters()`, se mueven juntos con `.to()` y se guardan con `state_dict()`. El cálculo se escribe en `forward()`.

```
class SpiralNet(nn.Module):
    def __init__(self, width=64, depth=3):
        super().__init__()
        blocks, fan_in = [], 2
        for _ in range(depth):
            blocks += [nn.Linear(fan_in, width), nn.Tanh()]
            fan_in = width
        self.body = nn.Sequential(*blocks)
        self.score = nn.Linear(width, 1)   # un logit por ejemplo

    def forward(self, x):
        return self.score(self.body(x))

probe_net = SpiralNet()
n_params = sum(p.numel() for p in probe_net.parameters())
print(probe_net)
print("parametros:", n_params)
```

```
SpiralNet(
  (body): Sequential(
    (0): Linear(in_features=2, out_features=64, bias=True)
    (1): Tanh()
    (2): Linear(in_features=64, out_features=64, bias=True)
    (3): Tanh()
    (4): Linear(in_features=64, out_features=64, bias=True)
    (5): Tanh()
  )
  (score): Linear(in_features=64, out_features=1, bias=True)
)
parametros: 8577
```

6.2. Capas propias: densa y *Dropout* desde cero

Implementar una capa enseña qué hay dentro de las capas estándar. `nn.Parameter` marca un tensor como parámetro entrenable del módulo. `register_buffer`, en cambio, guarda estado que **no** se entrena; aquí no hace falta.

```
class HandDense(nn.Module):
    """y = x W^T + b, con inicializacion de Kaiming uniforme."""

    def __init__(self, n_in, n_out):
        super().__init__()
        bound = 1.0 / np.sqrt(n_in)
        init = torch.empty(n_out, n_in).uniform_(-bound, bound)
        self.weight = nn.Parameter(init)
```

```
        self.bias = nn.Parameter(torch.zeros(n_out))

    def forward(self, x):
        return x @ self.weight.T + self.bias


class HandDropout(nn.Module):
    """Apaga unidades con probabilidad p solo en modo entrenamiento."""

    def __init__(self, p=0.2):
        super().__init__()
        self.p = p

    def forward(self, x):
        if not self.training or self.p == 0.0:
            return x
        keep = (torch.rand_like(x) > self.p).float()
        return x * keep / (1.0 - self.p)

layer = HandDropout(0.5)
sample = torch.ones(1, 8)
layer.train(); print("modo train:", layer(sample))
layer.eval();  print("modo eval: ", layer(sample))
```

```
modo train: tensor([[0., 2., 0., 2., 0., 0., 0., 0.]])
modo eval:  tensor([[1., 1., 1., 1., 1., 1., 1., 1.]])
```

6.3. `train()` y `eval()`

`model.train()` y `model.eval()` cambian el atributo `training` de todos los submódulos. Afectan a capas cuyo comportamiento depende del modo, como *Dropout* y *Batch Normalization*. **No** desactivan el cálculo de gradientes; para eso se usa `torch.no_grad()` o `torch.inference_mode()`.

7. Un bucle de entrenamiento reutilizable, al estilo Keras

Keras ofrece `fit`, `evaluate`, `predict`, un historial y *callbacks*. En PyTorch esas piezas se escriben explícitamente, lo que da control total. La clase siguiente las agrupa en unas 60 líneas.

```
class KerasLikeTrainer:
    def __init__(self, model, optimizer, loss_fn, l1=0.0):
        self.model, self.optimizer, self.loss_fn = model, optimizer, loss_fn
        self.l1 = l1
        self.history = {"loss": [], "acc": [], "val_loss": [], "val_acc": []}

    def _l1_penalty(self):
        return sum(p.abs().sum() for n, p in self.model.named_parameters()
                   if "weight" in n)

    def _run_epoch(self, loader, train):
        self.model.train(train)
        total, hits, count = 0.0, 0.0, 0
        with torch.set_grad_enabled(train):
            for xb, yb in loader:
```

```
                logits = self.model(xb)
                loss = self.loss_fn(logits, yb)
                if train:
                    full = loss
                    if self.l1:
                        full = loss + self.l1 * self._l1_penalty()
                    self.optimizer.zero_grad()
                    full.backward()
                    self.optimizer.step()
                total += loss.item() * len(yb)
                hits += ((logits >= 0).float() == yb).sum().item()
                count += len(yb)
        return total / count, hits / count

    def fit(self, train_loader, val_loader, epochs, patience=None,
            checkpoint=None, restore_best=True):
        best, waited, best_state = float("inf"), 0, None
        for _ in range(epochs):
            loss, acc = self._run_epoch(train_loader, train=True)
            val_loss, val_acc = self._run_epoch(val_loader, train=False)
            for key, val in zip(self.history, (loss, acc, val_loss, val_acc)):
                self.history[key].append(val)
            if val_loss < best - 1e-4:
                best, waited = val_loss, 0
                best_state = copy.deepcopy(self.model.state_dict())
                if checkpoint:
                    torch.save({"model": best_state,
                                "optimizer": self.optimizer.state_dict(),
                                "epoch": len(self.history["loss"])}, checkpoint)
            else:
                waited += 1
                if patience is not None and waited >= patience:
                    break
        if restore_best and best_state is not None:   # mejores pesos
            self.model.load_state_dict(best_state)
        return self.history

    def evaluate(self, loader):
        return self._run_epoch(loader, train=False)

    @torch.inference_mode()
    def predict_proba(self, features):
        self.model.eval()
        return torch.sigmoid(self.model(torch.as_tensor(features)))
```

Los puntos importantes del diseño:

- `zero_grad()` → `backward()` → `step()` es la secuencia de cada paso de entrenamiento.
- `torch.set_grad_enabled(train)` evita construir el grafo de gradientes al validar.
- El *early stopping* guarda una copia de los mejores pesos y los restaura al terminar.
- El punto de control (*checkpoint*) guarda el modelo **y el optimizador**. El estado de Adam (los promedios de los momentos) es necesario para reanudar el entrenamiento exactamente donde iba.

```
torch.manual_seed(11)
workdir = pathlib.Path(tempfile.mkdtemp(prefix="guia02_"))
net = SpiralNet(width=64, depth=3)
```

```
trainer = KerasLikeTrainer(net, torch.optim.Adam(net.parameters(), lr=3e-3),
                           nn.BCEWithLogitsLoss())
hist = trainer.fit(learn_loader, tune_loader, epochs=300, patience=40,
                   checkpoint=workdir / "mejor.pt")
val_loss, val_acc = trainer.evaluate(tune_loader)
print(f"epocas: {len(hist['loss'])} | val_loss={val_loss:.4f} | "
      f"val_acc={val_acc:.3f}")
```

```
epocas: 95 | val_loss=0.2564 | val_acc=0.913
```

```
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.8))
axes[0].plot(hist["loss"], label="entrenamiento")
axes[0].plot(hist["val_loss"], label="validación")
axes[0].set_xlabel("época"); axes[0].set_ylabel("BCE"); axes[0].legend()
axes[1].plot(hist["acc"], label="entrenamiento")
axes[1].plot(hist["val_acc"], label="validación")
axes[1].set_xlabel("época"); axes[1].set_ylabel("exactitud")
plt.tight_layout(); plt.show()
```

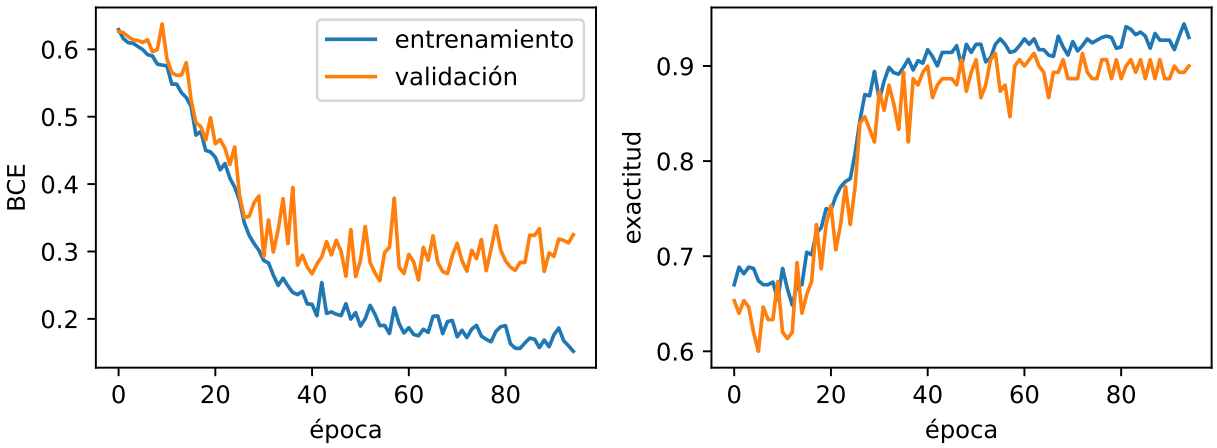


Figura 2: Pérdida y exactitud por época del MLP sobre las espirales.

7.1. La frontera de decisión

Para ver lo que aprendió la red, se evalúa la probabilidad en una malla fina del plano y se dibuja la curva de nivel $p = 0.5$.

```
def plot_boundary(trainer, ax, title):
    g = np.linspace(-2.6, 2.6, 250, dtype=np.float32)
    gx, gy = np.meshgrid(g, g)
    grid_pts = np.stack([gx.ravel(), gy.ravel()], axis=1)
    prob = trainer.predict_proba(grid_pts).numpy().reshape(gx.shape)
    ax.contourf(gx, gy, prob, levels=20, cmap="RdBu_r", alpha=0.55)
    ax.contour(gx, gy, prob, levels=[0.5], colors="black", linewidths=1.2)
    hx, hy = hold_set[:]
    ax.scatter(hx[:, 0], hx[:, 1], c=hy[:, 0], cmap="RdBu_r", s=8,
               edgecolors="white", linewidths=0.3)
    ax.set_title(title, fontsize=9); ax.set_aspect("equal")
    ax.set_xticks([]); ax.set_yticks([])
```

```
fig, ax = plt.subplots(figsize=(4.0, 4.0))
plot_boundary(trainer, ax, "MLP 3 x 64, tanh")
plt.tight_layout(); plt.show()
```

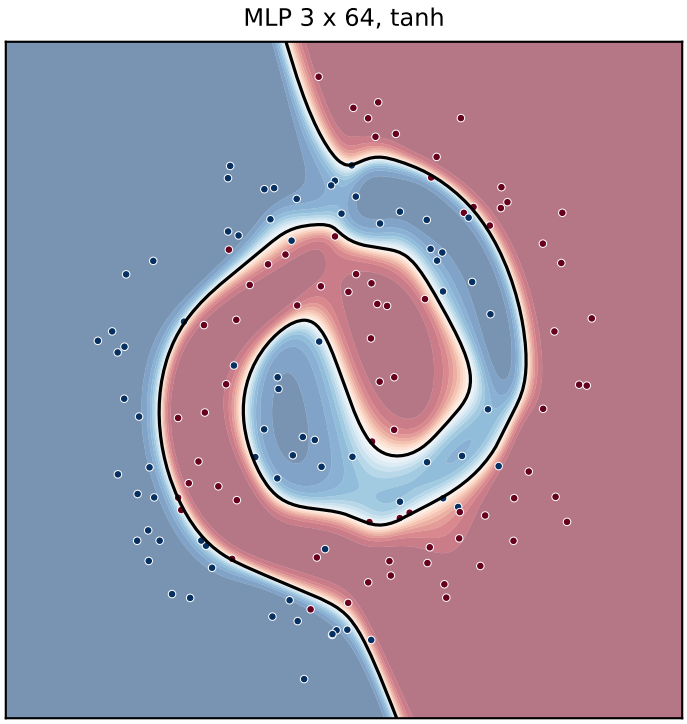


Figura 3: Probabilidad estimada de la clase 1 y frontera p = 0.5 (línea negra) sobre los datos de prueba.

8. Evaluación final y guardado

8.1. Prueba, matriz de confusión y métricas

La **exactitud** resume, pero no distingue tipos de error. La matriz de confusión cuenta verdaderos positivos (VP), falsos positivos (FP), falsos negativos (FN) y verdaderos negativos (VN). De ella salen la **precisión**, la **sensibilidad** o *recall* y la **F1**, su media armónica:

$$\text{precisión} = \frac{VP}{VP + FP}, \qquad \text{sensibilidad} = \frac{VP}{VP + FN}, \qquad F1 = \frac{2 \cdot \text{precisión} \cdot \text{sensibilidad}}{\text{precisión} + \text{sensibilidad}}.$$

```
hx, hy = hold_set[:]
pred = (trainer.predict_proba(hx) >= 0.5).float()
def count(p_val, y_val):
    return int(((pred == p_val) & (hy == y_val)).sum())

tp, fp, fn, tn = count(1, 1), count(1, 0), count(0, 1), count(0, 0)
precision, recall = tp / (tp + fp), tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
print(f"matriz de confusion  [[VN={tn:>3}  FP={fp:>3}]\n"
```

```
      f"                      [FN={fn:>3}  VP={tp:>3}]]")
print(f"exactitud={(tp + tn) / len(hy):.3f}  precision={precision:.3f}  "
      f"sensibilidad={recall:.3f}  F1={f1:.3f}")
```

```
matriz de confusion  [[VN= 68  FP=  7]
                      [FN=  8  VP= 67]]
exactitud=0.900  precision=0.905  sensibilidad=0.893  F1=0.899
```

8.2. Guardar y restaurar

Se recomienda guardar el `state_dict` (un diccionario de tensores) y no el objeto completo con *pickle*. Así el archivo no depende de la ruta exacta del código de la clase. Al restaurar, se crea la arquitectura y se cargan los pesos y, si se quiere seguir entrenando, el estado del optimizador.

```
bundle = torch.load(workdir / "mejor.pt")
clone = SpiralNet(width=64, depth=3)
clone.load_state_dict(bundle["model"])
clone_opt = torch.optim.Adam(clone.parameters(), lr=3e-3)
clone_opt.load_state_dict(bundle["optimizer"])
clone.eval()
with torch.inference_mode():
    same = torch.allclose(clone(hx), trainer.model(hx))
print("epoca del mejor punto de control:", bundle["epoch"])
print("predicciones identicas tras cargar:", same)
print("pasos registrados en el estado de Adam:",
      int(clone_opt.state_dict()["state"][0]["step"]))
```

```
epoca del mejor punto de control: 55
predicciones identicas tras cargar: True
pasos registrados en el estado de Adam: 1210
```

9. Regularización y normalización en PyTorch

9.1. L2, L1 y *Dropout*

- **L2.** Los optimizadores aceptan `weight_decay=` , que añade λw al gradiente. En `Adam` ese término se mezcla con la adaptación de la tasa. `AdamW` lo aplica **desacoplado**, que es la forma recomendada.
- **L1.** No existe un argumento equivalente. Se suma $\lambda_1 \sum |w|$ a la pérdida; el entrenador anterior lo hace con el parámetro `l1`.
- **Dropout.** Se intercala `nn.Dropout(p)` (o la capa `HandDropout`) entre capas.

Se compara su efecto con **solo 120 puntos** de entrenamiento, para provocar sobreajuste. Cada variante se entrena 400 épocas, sin *early stopping* (`restore_best=False`), con tres semillas, y se promedian los resultados. Así la comparación no depende de una inicialización afortunada. Se reportan la exactitud y la BCE, porque miden cosas distintas.

```
small_set = torch.utils.data.Subset(learn_set, range(120))
small_loader = DataLoader(small_set, batch_size=16, shuffle=True,
                          generator=torch.Generator().manual_seed(5))
```

```python
class RegNet(nn.Module):
    def __init__(self, drop=0.0, width=128):
        super().__init__()
        self.layers = nn.Sequential(
            HandDense(2, width), nn.ReLU(), HandDropout(drop),
            HandDense(width, width), nn.ReLU(), HandDropout(drop),
            HandDense(width, 1))

    def forward(self, x):
        return self.layers(x)

def run_variant(drop=0.0, decay=0.0, l1=0.0, seed=0):
    torch.manual_seed(seed)
    model = RegNet(drop)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=decay)
    tr = KerasLikeTrainer(model, opt, nn.BCEWithLogitsLoss(), l1=l1)
    tr.fit(small_loader, tune_loader, epochs=400, restore_best=False)
    (fit_loss, fit_acc) = tr.evaluate(small_loader)
    (val_loss, val_acc) = tr.evaluate(tune_loader)
    return fit_acc, val_acc, fit_loss, val_loss

variants = {"sin regularizar": {}, "L2 (AdamW 0.05)": {"decay": 0.05},
            "L2 (AdamW 0.5)": {"decay": 0.5}, "L1 (1e-4)": {"l1": 1e-4},
            "Dropout 0.3": {"drop": 0.3}}
print(f"{'variante':<16}{'exact.ent':>10}{'exact.val':>10}"
      f"{'BCE ent':>9}{'BCE val':>9}")
reg_scores = {}
for name, kw in variants.items():
    runs = np.array([run_variant(seed=s, **kw) for s in range(3)])
    fa, va, fl, vl = runs.mean(axis=0)
    reg_scores[name] = (fa, va, fl, vl)
    print(f"{name:<16}{fa:>10.3f}{va:>10.3f}{fl:>9.3f}{vl:>9.3f}")
```

variante	exact.ent	exact.val	BCE ent	BCE val
sin regularizar	0.975	0.811	0.047	1.156
L2 (AdamW 0.05)	0.989	0.793	0.054	0.854
L2 (AdamW 0.5)	0.936	0.789	0.223	0.474
L1 (1e-4)	0.986	0.838	0.062	0.632
Dropout 0.3	0.969	0.838	0.093	0.453

Cómo leer la tabla

Sin regularizar, la red casi memoriza los 120 puntos: su BCE de entrenamiento es mínima y la de validación es muchas veces mayor. El modelo da probabilidades extremas, cercanas a 0 o a 1, también cuando se equivoca. Las técnicas de regularización cambian poco la exactitud de validación, pero reducen claramente la BCE de validación: el modelo se vuelve **menos confiado en sus errores**. Si la penalización es demasiado fuerte, como L2 con 0.5, la exactitud empieza a bajar. Por eso la intensidad se elige comparando varias en validación.

9.2. *Batch*, *Layer* y *Group Normalization*

En PyTorch las tres existen como capas:

Capa	Normaliza sobre	Forma de entrada típica
`nn.BatchNorm1d(C)`	El lote, por cada característica	`(N, C)`
`nn.LayerNorm(C)`	Las características de cada ejemplo	`(N, …, C)`
`nn.GroupNorm(G, C)`	G grupos de características de cada ejemplo	`(N, C, …)`

```
gen = torch.Generator().manual_seed(0)
acts = 4.0 + 2.5 * torch.randn(6, 8, generator=gen)   # 6 ejemplos, 8 rasgos
layers = {"BatchNorm1d": nn.BatchNorm1d(8), "LayerNorm": nn.LayerNorm(8),
          "GroupNorm(2)": nn.GroupNorm(2, 8)}
for name, layer in layers.items():
    layer.train()
    out = layer(acts).detach()
    print(f"{name:<13} |media| max por rasgo={out.mean(0).abs().max():.3f}  "
          f"por ejemplo={out.mean(1).abs().max():.3f}")
bn = layers["BatchNorm1d"]
print("BatchNorm guarda medias moviles:", bn.running_mean[:3].numpy().round(3))
```

```
BatchNorm1d   |media| max por rasgo=0.000  por ejemplo=0.647
LayerNorm     |media| max por rasgo=0.806  por ejemplo=0.000
GroupNorm(2)  |media| max por rasgo=0.660  por ejemplo=0.000
BatchNorm guarda medias moviles: [0.312 0.239 0.388]
```

BatchNorm deja en cero la media de cada rasgo a lo largo del lote, y *LayerNorm* la de cada ejemplo. *GroupNorm* centra cada grupo de cuatro rasgos dentro de cada ejemplo. Por eso la media de cada ejemplo completo también queda en cero. *BatchNorm* además acumula `running_mean` y `running_var`, que usa en modo `eval()`. Las otras dos no guardan estado entre lotes.

10. PyTorch Lightning: el mismo modelo con menos código de infraestructura

Lightning separa **qué** se entrena de **cómo** se entrena. El `LightningModule` declara el modelo, el paso de entrenamiento y validación y el optimizador. El `Trainer` se encarga del bucle, los dispositivos, el registro de métricas, los puntos de control y el *early stopping*.

```
import lightning as L
from lightning.pytorch.callbacks import EarlyStopping, ModelCheckpoint

class SpiralClassifier(L.LightningModule):
    def __init__(self, width=64, depth=3, lr=3e-3):
        super().__init__()
        self.save_hyperparameters()
        self.net = SpiralNet(width, depth)
        self.loss_fn = nn.BCEWithLogitsLoss()

    def forward(self, x):
        return self.net(x)

    def _shared(self, batch, stage):
        xb, yb = batch
        logits = self(xb)
        loss = self.loss_fn(logits, yb)
```

```
        acc = ((logits >= 0).float() == yb).float().mean()
        self.log_dict({f"{stage}_loss": loss, f"{stage}_acc": acc},
                      on_epoch=True, on_step=False)
        return loss

    def training_step(self, batch, batch_idx):
        return self._shared(batch, "train")

    def validation_step(self, batch, batch_idx):
        self._shared(batch, "val")

    def test_step(self, batch, batch_idx):
        self._shared(batch, "test")

    def predict_step(self, batch, batch_idx):
        return torch.sigmoid(self(batch[0]))

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.hparams.lr)

L.seed_everything(11, verbose=False)
best_ckpt = ModelCheckpoint(dirpath=workdir / "lightning", monitor="val_loss",
                            mode="min", filename="espirales-{epoch:03d}")
lit_trainer = L.Trainer(max_epochs=300, accelerator="cpu", logger=False,
                        enable_progress_bar=False, enable_model_summary=False,
                        callbacks=[EarlyStopping("val_loss", patience=40),
                                   best_ckpt])
lit_model = SpiralClassifier()
lit_trainer.fit(lit_model, learn_loader, tune_loader)
print("epocas entrenadas:", lit_trainer.current_epoch)
print("mejor checkpoint:", pathlib.Path(best_ckpt.best_model_path).name)
```

```
epocas entrenadas: 96
mejor checkpoint: espirales-epoch=055.ckpt
```

```
restored = SpiralClassifier.load_from_checkpoint(best_ckpt.best_model_path)
val_metrics = lit_trainer.validate(restored, tune_loader, verbose=False)[0]
test_metrics = lit_trainer.test(restored, hold_loader, verbose=False)[0]
probs = torch.cat(lit_trainer.predict(restored, hold_loader))
print({k: round(v, 4) for k, v in {**val_metrics, **test_metrics}.items()})
print("predicciones:", tuple(probs.shape), "| primeras:",
      probs[:4, 0].numpy().round(3))
```

```
{'val_loss': 0.2487, 'val_acc': 0.9, 'test_loss': 0.271, 'test_acc': 0.8933}
predicciones: (150, 1) | primeras: [0.966 0.564 0.041 0.996]
```

`load_from_checkpoint` reconstruye el módulo con los hiperparámetros guardados por `save_hyperparameters()`. `validate`, `test` y `predict` reutilizan el mismo `Trainer`.

11. Keras 3 con *backend* PyTorch

Keras 3 es multi-*backend*: el mismo código de Keras puede ejecutarse sobre TensorFlow, JAX o PyTorch. El *backend* se elige con la variable de entorno `KERAS_BACKEND` **antes** de importar Keras; esta

guía la fijó en la primera celda. Con el *backend* `torch`, `fit()` acepta directamente los `DataLoader` de PyTorch.

```
import keras
keras.utils.set_random_seed(11)
print("backend activo:", keras.backend.backend())

keras_net = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(64, activation="tanh"),
    keras.layers.Dense(64, activation="tanh"),
    keras.layers.Dense(64, activation="tanh"),
    keras.layers.Dense(1),
])
keras_net.compile(optimizer=keras.optimizers.Adam(3e-3),
                  loss=keras.losses.BinaryCrossentropy(from_logits=True),
                  metrics=[keras.metrics.BinaryAccuracy(threshold=0.0,
                                                        name="acc")])
keras_hist = keras_net.fit(learn_loader, validation_data=tune_loader,
                           epochs=150, verbose=0,
                           callbacks=[keras.callbacks.EarlyStopping(
                               patience=40, restore_best_weights=True)])
k_loss, k_acc = keras_net.evaluate(hold_loader, verbose=0)
print(f"Keras/torch -> epocas={len(keras_hist.history['loss'])}  "
      f"test_loss={k_loss:.4f}  test_acc={k_acc:.3f}")
```

```
backend activo: torch
Keras/torch -> epocas=150  test_loss=0.2248  test_acc=0.940
```

`BinaryAccuracy(threshold=0.0)` aplica el umbral sobre los logits, porque la última capa no tiene sigmoide. Con `from_logits=True`, la pérdida aplica la sigmoide de forma numéricamente estable, igual que `BCEWithLogitsLoss`.

12. Comparación: PyTorch, Lightning y Keras

Aspecto	PyTorch puro	PyTorch Lightning	Keras 3
Bucle de entrenamiento	Se escribe a mano	Lo ejecuta `Trainer.fit()`	Lo ejecuta `model.fit()`
Definir el modelo	`nn.Module`	`nn.Module` dentro de un `LightningModule`	`Sequential`, funcional o subclase
Validación y prueba	Funciones propias	`validation_step`, `test_step`	`validation_data`, `evaluate()`
Early stopping, *checkpoints*	Código propio	*Callbacks* incluidos	*Callbacks* incluidos
Varias GPU o precisión mixta	Configuración manual	Argumentos del `Trainer`	Según el *backend*
Control fino	Máximo	Alto (*hooks*)	Medio (subclase de `Model`, `train_step`)
Curva de aprendizaje	Exige entender cada pieza	Exige PyTorch más convenciones	La más suave

Escribir primero el bucle a mano hace transparentes las abstracciones. Cuando se entiende lo que hacen `fit()` o `Trainer`, se pueden usar con confianza y depurar cuando fallan.

13. Dimensiones de tensores y flujo de datos

B es el tamaño del lote. El `DataLoader` de entrenamiento usa `batch_size=32`. Con 700 ejemplos hay $700 = 21 \cdot 32 + 28$, es decir, 22 lotes por época, y el último tiene $B = 28$. Validación y prueba usan `batch_size=256`, pero solo tienen 150 ejemplos, así que forman un único lote de $B = 150$.

`SpiralNet(width=64, depth=3)`, la red usada con el entrenador propio, con Lightning y, con la misma arquitectura, en Keras:

Etapa	Shape	Significado de cada dimensión	Parámetros
Entrada `xb`	$(B, 2)$	B puntos × 2 coordenadas	—
`Linear(2, 64)` + `Tanh`	$(B, 64)$	64 activaciones ocultas por punto	$2 \cdot 64 + 64 = 192$
`Linear(64, 64)` + `Tanh`	$(B, 64)$	64 activaciones ocultas por punto	$64 \cdot 64 + 64 = 4\,160$
`Linear(64, 64)` + `Tanh`	$(B, 64)$	64 activaciones ocultas por punto	$4\,160$
`score = Linear(64, 1)`	$(B, 1)$	Un *logit* por punto	$64 + 1 = 65$
Etiqueta `yb`	$(B, 1)$	0 o 1, en `float32`	—
`BCEWithLogitsLoss`	$()$	Promedio de las B pérdidas	—

Total: $192 + 2 \cdot 4\,160 + 65 = 8\,577$ parámetros. La red de regularización (`RegNet`, §9.1) usa una anchura de 128: $(B, 2) \to (B, 128) \to (B, 128) \to (B, 1)$, con $384 + 16\,512 + 129 = 17\,025$ parámetros. Las capas `HandDropout` no cambian la forma.

Convención de PyTorch. `nn.Linear(n_in, n_out)` guarda `weight` con forma (n_{out}, n_{in}) y calcula `x @ weight.T + bias`. Keras guarda el *kernel* al revés, (n_{in}, n_{out}). Al trasladar pesos entre bibliotecas hay que transponer.

Ejemplo de interpretación. En un lote completo, la primera capa calcula $(32, 2) \times (2, 64) = (32, 64)$: 2 048 activaciones a partir de 192 parámetros. Si las etiquetas tuvieran forma $(32,)$ en vez de $(32, 1)$, la resta entre *logits* y etiquetas se difundiría a una matriz $(32, 32)$, sin ningún error explícito. Por eso `PointCloud` aplica `unsqueeze(1)` a las etiquetas. El código siguiente lo comprueba con un lote real.

```
xb, yb = next(iter(learn_loader))
print("lotes de entrenamiento:", len(learn_loader),
      "| tamano del ultimo:", len(learn_set) - 32 * (len(learn_loader) - 1))
print(f"{'etapa':<24}{'shape':>10}{'parametros':>12}")
print(f"{'entrada xb':<24}{str(tuple(xb.shape)):>10}{'-':>12}")
h = xb
with torch.no_grad():
    for layer in trainer.model.body:
        h = layer(h)
        if isinstance(layer, nn.Linear):
            n_p = sum(p.numel() for p in layer.parameters())
            print(f"{'Linear ' + str(tuple(layer.weight.shape)):<24}"
                  f"{str(tuple(h.shape)):>10}{n_p:>12}")
    logits = trainer.model.score(h)
```

```
print(f"{'score (logits)':<24}{str(tuple(logits.shape)):>10}"
      f"{sum(p.numel() for p in trainer.model.score.parameters()):>12}")
loss = nn.BCEWithLogitsLoss()(logits, yb)
print(f"{'perdida':<24}{str(tuple(loss.shape)):>10}")
print("etiquetas (B, 1) ->", tuple((logits - yb).shape),
      "| etiquetas (B,) ->", tuple((logits - yb[:, 0]).shape))
```

```
lotes de entrenamiento: 22 | tamano del ultimo: 28
etapa                         shape  parametros
entrada xb                 (32, 2)           -
Linear (64, 2)            (32, 64)         192
Linear (64, 64)           (32, 64)        4160
Linear (64, 64)           (32, 64)        4160
score (logits)             (32, 1)          65
perdida                         ()
etiquetas (B, 1) -> (32, 1) | etiquetas (B,) -> (32, 32)
```

La primera columna muestra la forma de `weight` de cada capa, (n_{out}, n_{in}), y confirma la convención de PyTorch.

14. Hiperparámetros del modelo

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Datos	500 puntos por brazo, 1.75 vueltas, ruido 0.18	Dificultad del problema	Más ruido o más vueltas: clases más solapadas y menor exactitud alcanzable
Partición	70 / 15 / 15 % (700 / 150 / 150)	Ajuste, selección y evaluación final	Más datos de validación dan decisiones más estables
Tamaño de lote	32 (entrenamiento), 16 (regularización), 256 (evaluación)	Ejemplos por paso	Mayor: gradiente más estable y menos pasos. Menor: más ruido y más pasos por época
Tasa de aprendizaje	0.5 (regresión logística, SGD); $3 \cdot 10^{-3}$ (Adam o AdamW en las redes)	Tamaño del paso	Muy alta: inestabilidad. Muy baja: convergencia lenta
Épocas	40 (logística); hasta 300 (MLP y Lightning); 150 (Keras); 400 (regularización)	Presupuesto de entrenamiento	Más épocas sin regularizar favorecen el sobreajuste
Anchura y profundidad	64 × 3 capas (`SpiralNet`); 128 × 2 (`RegNet`)	Capacidad	Más capacidad sigue mejor la espiral, pero memoriza con pocos datos
Activación	`tanh` (`SpiralNet`), ReLU (`RegNet`)	No linealidad	Determina la forma de la frontera: suave o por tramos
Patience	40 épocas (entrenador propio, Lightning y Keras)	Espera antes de parar	Mayor: tolera mesetas. Menor: puede cortar antes de converger

Hiperparámetro	Valor usado	Función	Al aumentarlo / disminuirlo
Weight decay (L2)	0.05 y 0.5 (AdamW)	Encoge los pesos	Mayor: modelo más simple; 0.5 ya reduce la exactitud
L1	10^{-4}	Penaliza $\sum \lvert w \rvert$	Mayor: más pesos cercanos a cero y posible subajuste
Dropout	0.3	Apaga unidades al entrenar	Mayor: más regularización y entrenamiento más ruidoso
Umbral de decisión	0.5 sobre p (0 sobre el *logit*)	Convierte probabilidad en clase	Mayor: más precisión y menos sensibilidad (ejercicio 4)
Semillas	2026 global; 7 (partición); 3 (orden de lotes); 11 (modelos); 0, 1 y 2 (regularización)	Reproducibilidad	Cambian el resultado concreto, no el método

En términos de aprendizaje, la tasa y el tamaño de lote gobiernan la **convergencia**. La capacidad, las épocas, *weight decay*, L1 y *Dropout* gobiernan el equilibrio entre **sobreajuste y generalización**. *Patience* y el umbral son decisiones de **selección y operación** del modelo, no de su aprendizaje.

15. Errores frecuentes

1. **Olvidar `optimizer.zero_grad()`.** Los gradientes se acumulan entre pasos y las actualizaciones crecen sin control.
2. **Aplicar sigmoide y después `BCEWithLogitsLoss`.** La sigmoide queda aplicada dos veces; la pérdida recibe valores en $(0, 1)$ como si fueran logits.
3. **Etiquetas con forma `(N,)` frente a logits `(N, 1)`.** El *broadcasting* produce una matriz $N \times N$ sin error explícito. Hay que alinear las formas.
4. **Evaluar en modo `train()`.** *Dropout* sigue activo y *BatchNorm* usa estadísticas del lote: las métricas fluctúan.
5. **Etiquetas enteras.** La BCE espera etiquetas `float`; un tensor `long` genera un error de tipo.
6. **Guardar solo el modelo.** Para reanudar el entrenamiento también hace falta el estado del optimizador.
7. **Fijar `KERAS_BACKEND` después de importar Keras.** No tiene efecto: el *backend* se decide en la primera importación.

16. Ejercicios

1. **BCE a mano.** Con $\mathbf{x} = (2, -1)$, $\mathbf{w} = (0.3, 0.6)$, $b = -0.2$ y $y = 0$, calcula z, p, ℓ y $\nabla_{\mathbf{w}} \ell$. Verifica con `backward()`.
2. **Anchura y profundidad.** Entrena `SpiralNet` con `(width, depth)` igual a `(8, 1)`, `(64, 1)` y `(16, 4)`. Compara la frontera de decisión de cada una.
3. **Activaciones.** Sustituye `nn.Tanh` por `nn.ReLU` en `SpiralNet`. ¿Cómo cambia la forma de la frontera?
4. **Umbral.** En la prueba, cambia el umbral de 0.5 a 0.3 y a 0.7. ¿Qué ocurre con la precisión y la sensibilidad?

5. **Reanudar.** Carga el punto de control `mejor.pt` y entrena 20 épocas más. Comprueba que el contador `step` de Adam continúa desde el valor guardado.
6. **Lightning.** Añade a `SpiralClassifier` un programador de tasa de aprendizaje (`torch.optim.lr_scheduler.CosineAnnealingLR`) devuelto por `configure_optimizers`.

16.1. Respuestas orientativas

1. $z = 0.6 - 0.6 - 0.2 = -0.2$; $p = \sigma(-0.2) = 0.4502$; $\ell = -\log(1 - 0.4502) = 0.5981$; $\nabla_{\mathbf{w}} \ell = (p - y)\mathbf{x} = (0.9004,\ -0.4502)$.
2. Con `(8, 1)` la frontera es demasiado simple y la exactitud queda lejos de 1. Con `(64, 1)` mejora, pero necesita más épocas. Con `(16, 4)` la composición de capas sigue el giro de las espirales con menos parámetros.
3. Con ReLU la frontera se compone de tramos rectos (la red es lineal por partes). Con tanh es suave.
4. Bajar el umbral aumenta la sensibilidad y suele reducir la precisión. Subirlo tiene el efecto contrario.
5. El valor de `step` empieza en el número guardado y aumenta en un paso por lote.
6. `configure_optimizers` puede devolver:

```
{"optimizer": opt,
 "lr_scheduler": CosineAnnealingLR(opt, T_max=300)}
```

17. Glosario

Término	Significado
Logit	Puntuación real previa a la sigmoide
Entropía cruzada binaria	Pérdida $-[y \log p + (1-y)\log(1-p)]$
`autograd`	Motor de diferenciación automática de PyTorch
`nn.Module`	Clase base de toda capa o modelo en PyTorch
`state_dict`	Diccionario con los tensores de parámetros y *buffers*
`Dataset` / `DataLoader`	Acceso a ejemplos individuales y entrega en lotes
Early stopping	Detener el entrenamiento cuando la validación deja de mejorar
Checkpoint	Copia en disco del estado del entrenamiento
`LightningModule` / `Trainer`	Definición del modelo y motor de entrenamiento en Lightning
Backend de Keras	Biblioteca de cómputo sobre la que corre Keras 3

18. Conclusiones académicas

Qué se aprendió. La clasificación binaria se reduce a estimar una probabilidad con una sigmoide y a minimizar la entropía cruzada, cuyo gradiente respecto al *logit* es simplemente $p - y$. A partir de ahí la guía construyó cada pieza de PyTorch: `Dataset`, `DataLoader`, `autograd`, `nn.Module`, capas propias y un bucle de entrenamiento completo. Después las comparó con dos abstracciones de alto nivel, Lightning y Keras.

Qué demostraron los experimentos.

- La frontera lineal se quedó en una exactitud de validación de 0.647. Las espirales exigen una frontera no lineal.
- El MLP de 8 577 parámetros, con *early stopping*, se entrenó 95 épocas, con el mejor punto de control en la época 55. Alcanzó una exactitud de 0.913 en validación y de 0.900 en prueba, con F1 0.899.
- La versión Lightning se detuvo en la época 96 y obtuvo 0.893 de exactitud en prueba. La versión Keras sobre PyTorch, que en esta ejecución no activó el *early stopping* y completó 150 épocas, obtuvo 0.940. Esa diferencia no permite concluir que un *framework* sea superior. Las dos ejecuciones difieren en inicialización, número de épocas y criterio de restauración, y con 150 puntos de prueba la diferencia equivale a unos siete puntos.
- Con 120 puntos y sin regularizar, la BCE pasó de 0.047 en entrenamiento a 1.156 en validación: el modelo estaba sobreconfiado. Con *Dropout* 0.3 la BCE de validación bajó a 0.453. Las exactitudes de validación de las cinco variantes, en orden (0.811, 0.793, 0.789, 0.838, 0.838), cambiaron mucho menos que la BCE.

Cómo interpretar los resultados. La exactitud y la entropía cruzada responden preguntas distintas. La primera cuenta aciertos; la segunda mide si las probabilidades son creíbles. La regularización mejoró sobre todo la segunda, y eso importa cuando las probabilidades se usan para tomar decisiones con costos. Las comparaciones entre *frameworks* solo son justas si se controlan la semilla, el presupuesto de épocas y el criterio de parada.

Limitaciones.

- Problema bidimensional y sintético.
- Una sola partición de datos.
- Tres semillas en la comparación de regularización y una sola en los demás experimentos.
- Prueba de 150 puntos: una diferencia de 0.01 en exactitud equivale a menos de dos puntos mal clasificados.

Qué conviene profundizar.

- Validación cruzada y varias semillas para estimar la variabilidad.
- Calibración de probabilidades (diagramas de fiabilidad, *temperature scaling*).
- Clasificación multiclase con softmax (guía 03).
- Búsqueda de umbrales según el costo de cada tipo de error.

Relación con el *Deep Learning*. Los modelos de visión o lenguaje usan las mismas piezas: un *logit* por clase, entropía cruzada, `autograd`, módulos que se componen y un bucle de entrenamiento con validación y puntos de control. Saber escribir ese bucle a mano es lo que permite entender, y depurar, lo que automatizan Lightning o Keras en modelos con millones de parámetros.

19. Referencias

- Hortúa, H. J. *Deep Learning I — Clase 02: `02_Clasificacion.ipynb`*. Universidad El Bosque. Referencia conceptual de esta guía (enlaces en el recuadro de procedencia).
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press. Caps. 5–7.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Cap. 4.
- Loshchilov, I. y Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR.
- Documentación de PyTorch: https://pytorch.org/docs/stable/ · Lightning: https://lightning.ai/docs/pytorch/stable/ · Keras 3: https://keras.io/